

PRACTICAL 3

AIM : Implement algorithms to generate and verify message authentication codes (MACs) for ensuring data integrity and authenticity.

CLI CODE :

```
print("T120 DHEERAJ SINGH")
print()
import hmac
import hashlib

secret_key = input("Enter the secret key: ").encode()
message = input("Enter the message: ").encode()

mac = hmac.new(secret_key, message, hashlib.sha256).hexdigest()

print("\nGenerated MAC:", mac)

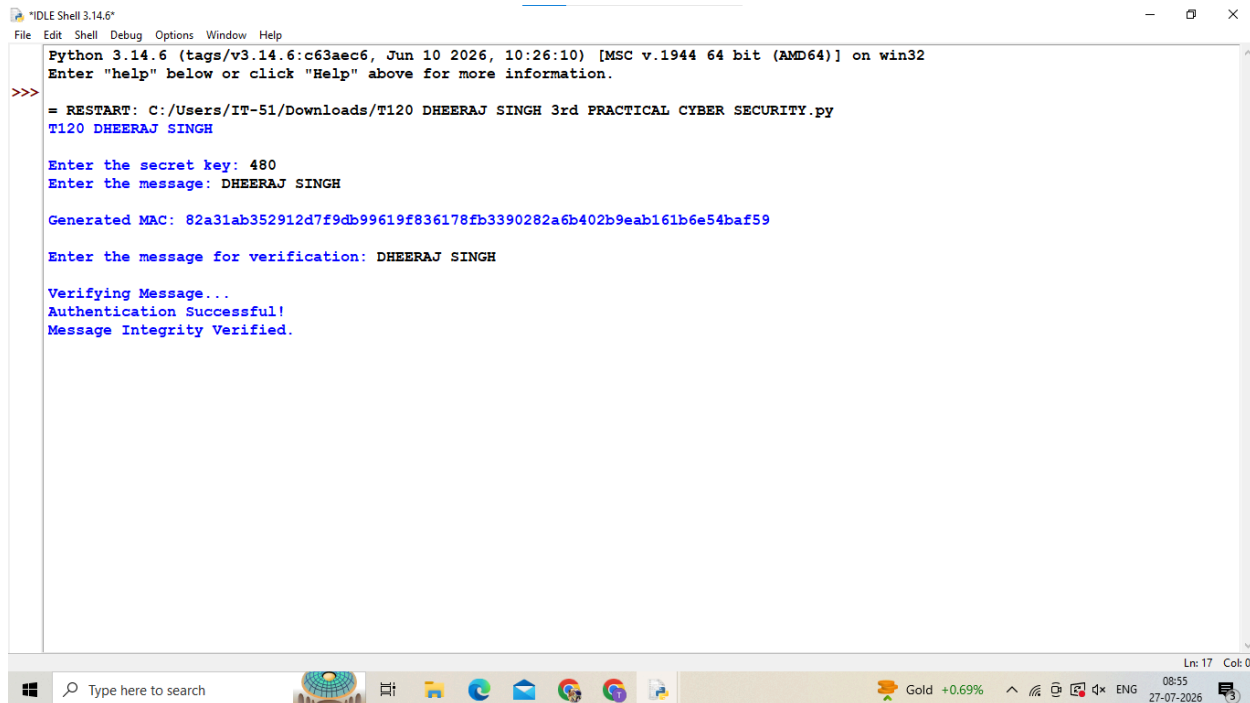
received_message = input("\nEnter the message for verification: ").encode()

received_mac = hmac.new(secret_key, received_message, hashlib.sha256).hexdigest()

print("\nVerifying Message...")

if hmac.compare_digest(mac, received_mac):
    print("Authentication Successful!")
    print("Message Integrity Verified.")
else:
    print("Authentication Failed!")
    print("Message has been modified.")
```

CLI OUTPUT :



```
"IDLE Shell 3.14.6"
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/IT-51/Downloads/T120 DHEERAJ SINGH 3rd PRACTICAL CYBER SECURITY.py
T120 DHEERAJ SINGH

Enter the secret key: 480
Enter the message: DHEERAJ SINGH

Generated MAC: 82a31ab352912d7f9db99619f836178fb3390282a6b402b9eab161b6e54baf59

Enter the message for verification: DHEERAJ SINGH

Verifying Message...
Authentication Successful!
Message Integrity Verified.
```

GUI CODE :

```
import tkinter as tk
from tkinter import messagebox
import hmac
import hashlib

def generate_mac():
    key = entry_key.get().encode()
    message = entry_message.get().encode()

    if not key or not message:
        messagebox.showerror("Error", "Please enter both key and message")
        return

    mac = hmac.new(key, message, hashlib.sha256).hexdigest()
    entry_mac.delete(0, tk.END)
    entry_mac.insert(0, mac)
```

```
def verify_mac():
    key = entry_key.get().encode()
    message = entry_verify_message.get().encode()
    mac_original = entry_mac.get()

    if not key or not message or not mac_original:
        messagebox.showerror("Error", "Please fill all fields")
        return

    new_mac = hmac.new(key, message, hashlib.sha256).hexdigest()

    if hmac.compare_digest(mac_original, new_mac):
        messagebox.showinfo("Result", "✅ Authentication
        Successful!\nMessage Integrity Verified.")
    else:
        messagebox.showerror("Result", "❌ Authentication Failed!\nMessage
        has been modified.")

root = tk.Tk()
root.title("MAC Generator & Verifier by T120 DHEERAJ SINGH")
root.geometry("500x400")

tk.Label(root, text="Secret Key").pack()
entry_key = tk.Entry(root, width=50)
entry_key.pack()

tk.Label(root, text="Message (Generate MAC)").pack()
entry_message = tk.Entry(root, width=50)
entry_message.pack()

tk.Button(root, text="Generate MAC", command=generate_mac,
bg="lightblue").pack(pady=5)

tk.Label(root, text="Generated MAC").pack()
entry_mac = tk.Entry(root, width=50)
```

```
entry_mac.pack()
```

```
tk.Label(root, text="Message (Verify)").pack()  
entry_verify_message = tk.Entry(root, width=50)  
entry_verify_message.pack()
```

```
tk.Button(root, text="Verify MAC", command=verify_mac,  
bg="lightgreen").pack(pady=10)
```

```
root.mainloop()
```

GUI OUTPUT :

